Exemple d’utilisation SQLi-Scanner-Pro

L'environnement virtuel (VENV)

```
┌──(root㉿urus)-[/home/urus/Bureau]
└─# python3 -m venv venv

┌──(root㉿urus)-[/home/urus/Bureau]
└─# source venv/bin/activate
```

Installer les dépendances

```
┌──(venv)─(root㉿urus)-[/home/urus/Bureau]
└─# pip install requests
Collecting requests
  Downloading requests-2.33.1-py3-none-any.whl.metadata (4.8 kB)
Collecting charset_normalizer<4,≥2 (from requests)
  Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)
Collecting idna<4,≥2.5 (from requests)
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,≥1.26 (from requests)
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi≥2023.5.7 (from requests)
  Downloading certifi-2026.2.25-py3-none-any.whl.metadata (2.5 kB)
Downloading requests-2.33.1-py3-none-any.whl (64 kB)
Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (215 kB)
Downloading idna-3.11-py3-none-any.whl (71 kB)
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
Downloading certifi-2026.2.25-py3-none-any.whl (153 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.2.25 charset_normalizer-3.4.7 idna-3.11 requests-2.33.1 urllib3-2.6.3
```

Cloner le dépôt

```
┌──(venv)─(root㉿urus)-[/home/urus/Bureau]
└─# git clone https://github.com/Maxime288/SQLi-Scanner-Pro.git
Clonage dans 'SQLi-Scanner-Pro' ...
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Réception d'objets: 100% (12/12), 6.41 Kio | 117.00 Kio/s, fait.
Résolution des deltas: 100% (1/1), fait.
```

Je me mets dans le dossier SQLi-Scanner-Pro **:**

```
┌──(venv)─(root㉿urus)-[/home/urus/Bureau]
└─# cd SQLi-Scanner-Pro
```

Je me mets dans le dossier cd sqli-scanner et je lance le script sur la machine virtuelle Metasploitable 2 :

```
┌──(venv)─(root㉿urus)-[/home/urus/Bureau/SQLi-Scanner-Pro]
└─# python sqli_scanner.py -u "http://192.168.1.134/mutillidae/index.php?page=user-info.php&username=1&password=&user-info-php-submit-button=View+Account+Details"

  ____  ___  _     _  ____
 / ___|/ _ \| |   (_) / ___|
| (___ | | | | |    _ | (___
 \___ \| | | | |   | | \___ \
 ___) | |_| | |___| | ___) |
|____/ \___\_\_____|_||____/
        SQL Injection Detector v1.0

[*] Analyse de l'URL : http://192.168.1.134/mutillidae/index.php?page=user-info.php&username=1&password=&user-info-php-submit-button=View+Account+Details
[*] Test du paramètre : page
[*] Test du paramètre : username

[VULNÉRABLE] Injection possible !
  Paramètre : username
  Payload   : '
  Type DB   : MySQL

Terminé en 0.98s
```